

Internal Working of insert() in Python List

Before understanding this method

A Python list works like a dynamic array. Internally it has two important ideas:

- **size** - number of elements currently present in the list.
- **capacity** - extra internal space reserved by Python for future elements.

When capacity is full, Python creates a bigger internal array, copies old elements, and then performs the operation.

Meaning

insert() adds one element at a specific index.

```
a = [10, 20, 30, 40]
a.insert(2, 25)
print(a)    # [10, 20, 25, 30, 40]
```

Internal steps

- Python checks the index where the new element has to be inserted.
- Python checks whether extra capacity is available.
- If capacity is full, Python creates bigger memory and copies old elements.
- Elements from the insertion index are shifted one position to the right.
- The new value is placed at the required index.
- The size of the list is increased by 1.

Before insert at index 2:

```
Index:  0   1   2   3
List:  10  20  30  40
```

Shift right:

```
Index:  0   1   2   3   4
List:  10  20  _  30  40
```

Place 25:

```
List:  10  20  25  30  40
```

Why insert can be slow

If you insert at the beginning, every element must move one step right.

```
a = [10, 20, 30, 40]
a.insert(0, 5)
# 10, 20, 30, 40 all shift right
# Result: [5, 10, 20, 30, 40]
```

Time complexity

- Insert at the end: almost $O(1)$ average.
- Insert at beginning or middle: $O(n)$, because shifting is required.
- General worst-case time complexity: $O(n)$.

Key point

`insert()` is slower than `append()` when used at the beginning or middle because elements must shift right.